QA.GURU · PYTHON · MOBILE 2

Мобильная автоматизация

Запускаем тесты Wikipedia на эмуляторе, телефоне и в облаке. Добавляем Allure.

pytest · Appium · Android · iOS

СЕГОДНЯ НА ЗАНЯТИИ

Запуск теста, устройства и отчёт

01

Запускаем тест

Выбираем сервер и устройство. Создаём и закрываем сессию.

02

Разбираем падение

Сохраняем экран, XML и видео. Открываем их в Allure.

03

Готовим iOS

Разбираемся с Simulator, сборкой приложения и WDA.

УЧЕБНЫЙ ПРОЕКТ

Что уже есть в python-appium

Код проекта

- Тест поиска в Wikipedia.
- Настройки сервера и драйверов.
- Выбор Android-устройства по UDID.
- Screenshot, XML и видео в Allure.

Разберём на уроке

- Как запускать тест на разных устройствах.
- Когда снимать вложения.
- Чем iOS отличается от Android.

iOS-драйверы уже есть. Локаторы самого теста пока только Android.

КАК РАБОТАЕТ APPIUM

Путь команды от теста до приложения

pytest
Тест и фикстуры

→

Python client
Команды драйвера

→

Appium
Сервер и драйвер

→

Устройство
Приложение

Android: UiAutomator2 управляет приложением.

iOS: XCUITest работает через WebDriverAgent.

Inspector — тоже клиент Appium. Сервер он не заменяет.

МЕСТО ЗАПУСКА

Выбираем DEVICE_HOST

ЗНАЧЕНИЕ	ГДЕ СОЗДАЁТСЯ СЕССИЯ
`emulator`	Локальный Android Emulator
`real`	Подключённый Android-телефон
`selenoid`	Учебная Android-ферма
`browserstack`	BrowserStack App Automate
`ios_simulator` / `ios_real`	Simulator / iPhone на Mac

Для iOS после настройки драйвера ещё нужно перенести сценарий поиска.

На телефоне и ферме — разные сборки

Локальный Android

- Stable **50611-r-2026-10-07**.
- Package: `org.wikipedia`.
- Путь к APK на машине сервера.
- Версия и SHA-256 записаны в README.

Официальный APK

Selenoid

- Sample Wikipedia 2017 года.
- Package: `org.wikipedia.alpha`.
- Ферма скачивает APK по URL.
- Экран поиска отличается от stable.

> Не копируем локаторы между сборками без проверки в Inspector.

ПЕРЕД УСТАНОВКОЙ

Версии для учебного окружения

Основные инструменты

- Python **3.14**.
- JDK **21**.
- Node.js **26**, npm 10+.

Python-пакеты устанавливаем из requirements.txt.

Сервер и драйверы

Appium	3.5.0
UiAutomator2	6.0.0
XCUITest	12.8.0
Allure CLI 2	2.34.1

Устанавливаем зависимости проекта

macOS / Linux

```
python3.14 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pytest --collect-only tests
```

Windows · PowerShell

```
py -3.14 -m venv .venv
.\.venv\Scripts\python.exe -m pip `
    install -r requirements.txt
.\.venv\Scripts\pytest.exe `
    --collect-only tests
```

С --collect-only pytest находит тесты, но не запускает их.

Что установить в Android Studio

SDK Manager

- SDK Platform.
- Platform-Tools: команда adb.
- Build-Tools и Command-line Tools.
- Emulator и system image.

Device Manager

- Создать AVD.
- Выбрать версию Android.
- Подобрать архитектуру образа.
- Дождаться полного старта системы.

Для Apple Silicon выбираем arm64. APK должен поддерживать тот же ABI.

Указываем пути к Java и Android SDK

```
export JAVA_HOME=$(/usr/libexec/java_home -v 21)
export ANDROID_HOME="$HOME/Library/Android/sdk"
export PATH="$JAVA_HOME/bin:$PATH"
export PATH="$ANDROID_HOME/platform-tools:$PATH"
export PATH="$ANDROID_HOME/emulator:$PATH"

java -version
adb version
```

Слева — пример для macOS.

На Linux укажите свой JDK и путь к SDK из Android Studio.

Не хватит просто задать PATH: проверьте, какую Java и adb он находит.

Настройка переменных в PowerShell

```
$sdk = "$env:LOCALAPPDATA\Android\Sdk"
$env:ANDROID_HOME = $sdk
$env:JAVA_HOME = "C:\Tools\jdk-21"
$env:Path += ";$env:JAVA_HOME\bin"
$env:Path += ";$sdk\platform-tools;$sdk\emulator"

java -version
adb version
```

Путь к JDK замените своим.

Путь к SDK проверьте в Android Studio.

После изменения постоянных переменных откройте новый терминал.

Appium и драйвер устанавливаем отдельно

```
npm install -g appium@3.5.0
appium driver install uiautomator2@6.0.0

appium --version
appium driver list --installed
appium driver doctor uiautomator2
```

UiAutomator2 управляет Android.

XCUITest нужен для iOS на Mac.

Установка Python-клиента не запускает Appium Server.

К какому серверу подключаемся

Параметры подключения

- Host и port сервера.
- SSL и base path.
- Options выбранного устройства.

Те же настройки должны быть в APPIUM_URL теста.

Частая ошибка: base path

В Appium 2 и 3 по умолчанию используется `/`.

Если сервер настроен на `/wd/hub`, этот путь нужен и клиенту.

> Версия Appium сама по себе не определяет base path.

Сначала запускаем AVD, потом тест

```
adb devices -l
adb -s "$UDID" shell getprop sys.boot_completed
adb -s "$UDID" shell getprop ro.product.cpu.abi
```

- AVD запущен в Device Manager.
- adb показывает статус device.
- Система полностью загрузилась.
- APK подходит по ABI и версии Android.

UDID берём из adb devices. При нескольких устройствах выбираем его явно.

Подключаем телефон по USB

1. Включить Developer Options.
2. Включить USB debugging.
3. Подтвердить доверие компьютеру.
4. Проверить adb devices -l.

```
options.udid = device_udid
options.app = apk_path
```

deviceName не заменяет UDID: выбираем конкретный телефон или эмулятор.

В проекте Android запускается с noReset=False. Для практики лучше учебный телефон.

Минимальная настройка сессии

```
options = UiAutomator2Options()
options.platform_name = "Android"
options.automation_name = "UiAutomator2"
options.udid = device_udid
options.app = apk_path

driver = webdriver.Remote(
    appium_url, options=options
)
```

appium_url — адрес сервера.

device_udid — serial из adb.

apk_path — файл на сервере.

Package и activity должны относиться к выбранному APK.

СТРУКТУРА ПРОЕКТА

У каждого файла своя задача

```
config.py
    читает настройки

drivers.py
    создаёт сессию

conftest.py
    готовит и закрывает драйвер

helpers/attach.py
    сохраняет вложения
```

Тест описывает поиск и проверку результата.

Переключение сервера не должно переписывать весь тест.

> Платформы могут иметь разные экраны и локаторы. Общим остаётся сценарий.

Ждём появления поля поиска

```
field = open_search_field(
    driver, config.timeout
)
field.send_keys("Appium")

assert wait.until(
    lambda drv: has_elements(drv, RESULT_LOCS)
)
```

- Ждём видимый элемент.
- Учитываем onboarding и окно Search widget.
- Проверяем результат поиска.

Следующий шаг: открыть нужную статью и проверить её заголовок.

Почему упал тест?

Последний экран.
Дерево элементов.
Видео действий.

Сохраним их в отчёте рядом с ошибкой.

Пишем шаги, сохраняем результаты

```
with allure.step("Search for Appium"):
    field.send_keys("Appium")

with allure.step("Verify results"):
    assert wait.until(
        lambda drv: has_elements(
            drv, RESULT_LOCS
        )
    )
```

```
pytest tests
allure generate allure-results \
    -o allure-report
```

Плагин allure-pytest создаёт результаты. Allure CLI строит из них отчёт.

Установка CLI и отдельные каталоги прогонов — в README.

Screenshot показывает экран, XML — элементы

```
allure.attach(
    driver.get_screenshot_as_png(),
    name="Screenshot",
    attachment_type=allure.attachment_type.PNG,
)
allure.attach(
    driver.page_source,
    name="Page source",
    attachment_type=allure.attachment_type.XML,
)
```

Screenshot: какой экран открыт?

XML: какие элементы видит драйвер?

Оба вложения снимаем до quit().

Закрываем драйвер даже после падения

```
session_id = drv.session_id
try:
    yield drv
finally:
    try:
        attach.screenshot(drv)
        attach.page_source(drv)
    finally:
        drv.quit()
    attach.video(config, session_id)
```

1. Сохраняем session_id.
2. Снимаем экран и XML.
3. Закрываем сессию.
4. Скачиваем видео фермы.

Локальную запись останавливаем ещё до quit().

Сохраняем экран до очистки состояния

```
@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when == "call" and report.failed:
        drv = item.funcargs.get("driver")
        if drv is not None:
            attach.screenshot(drv, "failure")
```

Hook срабатывает после теста, но до teardown.

Если упал setup, экран сохраняется отдельно в фикстуре.

Проверим на намеренно неверном assert.

Источники вложений

ГДЕ ЗАПУСКАЕМ	ЧТО ПОЛУЧАЕМ	ВИДЕО
Локальный Android	PNG, XML, запись экрана	До quit
Локальный iOS	PNG, XML, запись при поддержке	До quit
Selenoid	PNG, XML, MP4 фермы	После quit
BrowserStack	PNG, XML, видео и ссылка на сессию	После quit

Логи Appium и logcat пока не собираются автоматически.

Как ферма выбирает Android-контейнер

```
options.set_capability("browserName", "android")
options.set_capability("browserVersion", version)
options.app = apk_url
options.set_capability(
    "selenoid:options",
    {"enableVNC": True, "enableVideo": True},
)
```

- Версия берётся из каталога фермы.
- APK скачивается по URL.
- VNC показывает экран.
- enableVideo включает запись.

По умолчанию: Android 16.0.

Сначала quit(), потом MP4

Запись
enableVideo

→

Сессия
session_id

→

quit()
Завершение записи

→

MP4
Скачивание файла

```
video_path = f"/video/{session_id}.mp4"
allure.attach(
    video_bytes, name="Video",
    attachment_type=allure.attachment_type.MP4,
)
```

Helper ждёт появления файла, но не бесконечно.

Облачный запуск из первого занятия

```
DEVICE_HOST=browserstack pytest tests
```

Данные доступа и BROWSERSTACK_APP задаём в .env.

- Приложение загружается в App Automate.
- Сессия получает свой session_id.
- По нему API возвращает видео.

App Store приложение на вашем iPhone не заменяет загрузку IPA в облако.

Два способа подготовить Wikipedia

Simulator: готовим .app.
iPhone: ставим приложение из App Store.

Для управления iPhone отдельно настраиваем WDA.

Mac, Xcode и XCUITest

```
xcode-select -p
xcodebuild -version
xcrun simctl list devices available

appium driver install xcuitest@12.8.0
appium driver doctor xcuitest
```

- Нужен полный Xcode.
- Нужен установленный runtime.
- Версии Xcode и iOS должны подходить драйверу.

Command Line Tools не заменяют полный Xcode.

Для Simulator нужна своя .app

Simulator

- Сборка для iphonesimulator.
- .app или архив с .app.
- Архитектура под ваш Mac.
- Платная Developer-учётка не нужна.

Настоящий iPhone

- Установленное App Store приложение.
- Или подписанная device-сборка.
- UDID конкретного телефона.
- Подготовленный WDA.

Готовим Simulator-приложение до урока

```
xcodebuild \
  -project Wikipedia.xcodeproj \
  -scheme Wikipedia \
  -sdk iphonesimulator \
  -destination 'generic/platform=iOS Simulator' \
  -derivedDataPath ./build \
  CODE_SIGNING_ALLOWED=NO
```

- Берём официальные исходники.
- Выбираем конкретную версию.
- Проверяем зависимости сборки.
- Передаём студентам готовую .app.

Полная команда и настройки — в README.

Настраиваем XCUITestOptions

```
options = XCUITestOptions()
options.platform_name = "iOS"
options.automation_name = "XCUITest"
options.udid = simulator_udid
options.app = simulator_app_path

driver = webdriver.Remote(
    appium_url, options=options
)
```

UDID берём из simctl.

IOS_APP указывает на .app.

В конфигурации используем DEVICE_HOST=ios_simulator.

DEVICE_NAME и PLATFORM_VERSION задаём для своего runtime.

Подключаем устройство и подписываем WDA

1. Подключить iPhone к Mac.
2. Выполнить pairing в Xcode.
3. Подтвердить доверие компьютеру.
4. Включить Developer Mode и UI Automation.
5. Настроить Team и подпись WDA.

```
appium driver run xcuitest open-wda
```

WDA — отдельное приложение для управления UI.

Personal Team подходит для ручной настройки, но профиль имеет ограничения и срок действия.

Запускаем установленное приложение по bundleId

```
options.udid = iphone_udid
options.bundle_id = "org.wikimedia.wikipedia"
options.no_reset = True

driver = webdriver.Remote(
    appium_url, options=options
)
```

- Wikipedia уже установлена.
- WDA настроен для этого iPhone.
- Пересборка Wikipedia не нужна.

Не передаём app: установка из файла в этом примере не нужна.

КОГДА ЧТО-ТО НЕ РАБОТАЕТ

С чего начать проверку

ЧТО ПРОИЗОШЛО	ЧТО ПРОВЕРИТЬ
Нет сессии	Сервер, base path, driver и доступ
adb: unauthorized / offline	Доверие на телефоне, кабель, загрузку Android
Не установилось приложение	Путь или URL, ABI, SDK, подпись
Не найден элемент	Screenshot, XML, onboarding, версию приложения
xcodebuild: 65 / 70	Подпись WDA и выбранный Xcode/iOS
Нет видео	Запись, quit, публикацию MP4 и доступ

ПОРЯДОК ПРАКТИКИ

Показываем на одном Android-тесте

Сначала локально

1. Находим элемент в Inspector.
2. Запускаем поиск на AVD.
3. Вызываем контролируемое падение.
4. Разбираем Allure.

Затем меняем устройство

- Выбираем телефон по UDID.
- Запускаем на Selenoid.
- Сравниваем видео и экран.

Для iOS отдельно показываем настройку драйвера и перенос локаторов.

ДОМАШНЕЕ ЗАДАНИЕ

Добавьте проверки и инструкцию запуска

Android

- 2–4 проверки Wikipedia.
- Локальный и один удалённый запуск.
- Шаги, screenshot и XML в Allure.
- Видео там, где оно поддерживается.
- Инструкция без ключей и паролей.

Дополнительно

- Запуск на настоящем Android.
- iOS-локаторы и onboarding.
- Тест на Simulator или iPhone.

Проверяйте найденную статью, а не просто наличие любых элементов.

ПЕРЕД ЗАНЯТИЕМ

Что проверить преподавателю

Устройства и приложение

- AVD уже загружен.
- APK скачан и проверен.
- UDID выбран правильно.
- Ферма доступна.
- Для iOS готовы .app или WDA.

Тест и отчёт

- Есть успешный пробный прогон.
- Есть понятный пример падения.
- Вложения открываются в Allure.
- Сессия закрывается после ошибки.
- В отчёте нет секретов и личных данных.

ССЫЛКИ

Код и документация

- Учебный python-appium
- Требования Appium
- UiAutomator2
- Установка Inspector
- Совместимость XCUITest
- Подпись WebDriverAgent
- Примеры iOS capabilities
- Исходники Wikipedia iOS
- BundleId Wikipedia у Apple
- Запасное приложение: WDIO demo

WDIO demo даёт APK и готовую Simulator .app. Сборки для настоящего iPhone в релизе нет.